Table of contents

Chapter 4. Pretty URLs: a reverse proxy + local DNS

> **The payoff of this chapter:** type a bare `pihole.home` or `abs.home` in any browser on your tailnet (at home or on cellular) and land on the right service over HTTPS, no port numbers, no IP addresses, no scheme to type. And a pattern you'll reuse for every service you add from here on.

Right now you reach your two services awkwardly: `http://192.168.1.50/admin` for Pi-hole, `http://homelab:13378` for Audiobookshelf, an IP here, a port there. This part introduces the machinery that gives **every** service one clean name, starting with these two. When we add the dashboard in Chapter 5, it'll slot into the same system in three lines.

> **!** Why not `pihole.com`?
>
> A `.com` (or any public) name belongs to whoever registered it on the internet, you can't point `pihole.com` at your Pi any more than you can point `google.com` at it. We use a **private** suffix, `.home`, that exists only on *your* network.

How this works: two jobs, two tools

A working URL needs two separate things:

1. **The name has to resolve to your Pi.** `pihole.home` must turn into the Pi's address, that's **DNS** (we add records to Pi-hole and let Tailscale carry them to every device).
2. **Traffic hitting the Pi has to reach the right service.** `pihole.home` and `abs.home` both arrive on the Pi's web ports (80/443), but Pi-hole and Audiobookshelf listen on *different* ports. Something must read the requested name and forward to the correct backend, a

reverse proxy (we'll use **Caddy**, the simplest to configure). Caddy also gives each name an HTTPS certificate from its own private CA, so the URLs get a real padlock.

```
pihole.home   DNS   the Pi (443)   Caddy routes by name   pihole:80
abs.home      DNS   the Pi (443)   Caddy routes by name   audiobookshelf:80
```

Part A: Wire the services onto one network

Step 1: Create the shared Docker network and put Audiobookshelf on it

Caddy reaches each backend by **container name** (`pihole`, `audiobookshelf`), which only works if they share a Docker network. Create it:

```
docker network create homelab
```

Both Audiobookshelf (Chapter 2) and Pi-hole (Chapter 3) are **Compose projects**, so we attach each to the network the *declarative* way, by adding it to the project's `compose.yaml` (see the callout for why that's the only attachment that sticks). Start with Audiobookshelf: add the `networks` blocks to `~/audiobookshelf/compose.yaml`:

```
cat > ~/audiobookshelf/compose.yaml <<'EOF'
services:
  audiobookshelf:
    container_name: audiobookshelf
    image: ghcr.io/advplyr/audiobookshelf:latest

    ports:
      - "13378:80"

    volumes:
      - ./media/Audiobooks:/audiobooks
      - ./config:/config
      - ./metadata:/metadata

    networks:
      - homelab            # NEW: join the shared proxy network

    restart: unless-stopped

networks:
  homelab:
    external: true         # NEW: the shared network created above
EOF
( cd ~/audiobookshelf && docker compose up -d --force-recreate )
```

Any container on the `homelab` network can now reach the others by name, no IP addresses needed between containers.

⚠ Attach Compose services in the Compose file, not with `docker network connect`

It's tempting to run `docker network connect homelab audiobookshelf` instead, but it won't survive. Any time you recreate the container (`docker compose up --force-recreate`, or an update), **a recreate rebuilds it with only the networks declared in its `compose.yaml`**, a network you attached by hand is silently dropped, and Caddy would then fail to resolve the backend. So for every Compose service we add the `homelab` network *inside* the Compose file, where a recreate always re-attaches it. Pi-hole gets the same treatment in Step 2.

Step 2: Free port 80 and extend Pi-hole onto the tailnet

Three changes to Pi-hole's `~/pihole/compose.yaml`, all needed before the proxy works:

- **Free host port 80.** Caddy will be the single front door on port 80, so move Pi-hole's web UI to `8081`.
- **Listen on the tailnet, not just the LAN.** In Chapter 3 you bound Pi-hole to your LAN IP because it was a home-only service. To make `.home` names (and Pi-hole's own ad-blocking) work *everywhere*, Pi-hole's DNS must also answer on the Pi's Tailscale interface, so drop the `${PIHOLE_IP}:` prefix and let it listen on all interfaces.
- **Join the `homelab` network declaratively** (per the Step 1 callout) so Caddy can reach it as `pihole:80`, and so the recreate below, and every future `docker compose up`, re-attaches it automatically.

Replace `~/pihole/compose.yaml` with this updated version (the `ports:` and new `networks:` blocks are the only changes from Chapter 3):

```
cat > ~/pihole/compose.yaml <<'EOF'
services:
  pihole:
    container_name: pihole
    image: pihole/pihole:latest

    ports:
      - "53:53/tcp"          # all interfaces (LAN + tailnet); was
${PIHOLE_IP}:53:53
      - "53:53/udp"
      - "8081:80/tcp"        # web UI moved off 80, freeing it for Caddy;
was ${PIHOLE_IP}:80:80

    environment:
      TZ: "${TZ}"
      FTLCONF_webserver_api_password: "${PIHOLE_PASSWORD:?set
PIHOLE_PASSWORD in .env}"
      FTLCONF_dns_upstreams: "1.1.1.1;1.0.0.1"
      FTLCONF_dns_listeningMode: "ALL"
```

```
    volumes:
      - ./etc-pihole:/etc/pihole

    networks:
      - homelab            # NEW: declarative attachment, survives every
recreate

    restart: unless-stopped

networks:
  homelab:
    external: true         # NEW: the shared network you created in Step 1
EOF
```

Then recreate the container so the new ports and network take effect:

```
cd ~/pihole && docker compose up -d --force-recreate
```

> **Is listening on all interfaces safe?**
>
> Yes, in this setup. The only networks that can reach the Pi are your home LAN (behind the router) and your tailnet (authenticated WireGuard). The one unbreakable rule, same as Chapter 3: **never port-forward port 53 (or 80) from your router.** As long as you don't, "all interfaces" just means "my LAN and my tailnet," which is exactly who should be able to use it.

Pi-hole's admin UI now lives at `http://192.168.1.50:8081/admin` (your Pi's LAN IP), but in a moment you'll reach it as `https://pihole.home` instead.

Part B: Deploy the proxy and DNS

Step 3: Deploy Caddy

Caddy is its own small stack:

```
mkdir -p ~/proxy && cd ~/proxy
```

Create `~/proxy/Caddyfile`, the entire routing table:

```
cat > Caddyfile <<'EOF'
# `tls internal` makes Caddy issue each .home name a certificate from its
OWN
# built-in certificate authority (no public CA issues certs for a private
TLD).
```

```
# Caddy then serves HTTPS on 443 and auto-redirects http:// -> https://.
Once you
# trust Caddy's local CA on your devices (Step 7), a bare `pihole.home`
typed in
# the browser just works: no scheme, no warning, real padlock.

pihole.home {
    tls internal
    redir / /admin 302          # land on the admin page directly
    reverse_proxy pihole:80
}

abs.home {
    tls internal
    reverse_proxy audiobookshelf:80
}
EOF
```

Create `~/proxy/compose.yaml`:

```
cat > compose.yaml <<'EOF'
services:
  caddy:
    container_name: caddy
    image: caddy:latest
    ports:
      - "80:80"                 # the single front door (redirects to 443)
      - "443:443"               # HTTPS, on all interfaces (LAN + tailnet)
      - "443:443/udp"           # HTTP/3
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - homelab
    restart: unless-stopped

networks:
  homelab:
    external: true              # the shared network from Step 1

volumes:
  caddy_data:
  caddy_config:
EOF
```

Start it:

```
docker compose up -d
docker compose logs --tail 20
```

Caddy listens on `0.0.0.0:80` and `0.0.0.0:443`, all interfaces, including the Pi's Tailscale `100.x.y.z`, and reaches each backend by container name over the `homelab` network, so it ignores host port mappings entirely. That's why moving Pi-hole's *host* web port to 8081 doesn't bother Caddy: it talks to `pihole:80` inside the network. On first start, `tls internal` makes Caddy generate a local certificate authority and sign a cert for each `.home` name; you'll trust that CA on your devices in Step 7.

Step 4: Add the `.home` names to Pi-hole's DNS

Make the names resolve to the Pi. In the Pi-hole admin UI (`http://192.168.1.50:8081/admin` for now), go to **Settings → Local DNS Records** and add one **A record** per name, all pointing at the Pi's **Tailscale** IP (`tailscale ip -4` on the Pi):

Domain	IP
`pihole.home`	`100.x.y.z`
`abs.home`	`100.x.y.z`

> **Why the *Tailscale* IP, not the LAN IP**
>
> A DNS record holds one address. Pointing these at the Pi's `100.x.y.z` tailnet address makes them reachable from *anywhere* a device is on your tailnet, home Wi-Fi or cellular alike. Caddy is listening on that address (Step 3), so the traffic lands. (This assumes Tailscale is running on the device, which is the premise of the whole series.)

Step 5: Make Pi-hole your tailnet's DNS

For `*.home` to resolve on every device, your devices must ask **Pi-hole** for DNS. In the Tailscale admin console (https://login.tailscale.com), **DNS** page:

1. **Add nameserver → Custom**, enter the Pi's Tailscale IP (`100.x.y.z`), save.
2. Turn on **Override local DNS**. Now every tailnet device resolves through Pi-hole, so `*.home` names work, *and* you get Pi-hole ad-blocking on every device, even on cellular.

> **Want only `.home` through Pi-hole, not all your DNS?**
>
> Use **split DNS** instead: in **Add nameserver → Custom**, enable **Restrict to domain** and set the domain to `home`. Then only `*.home` lookups go to Pi-hole and everything else uses each device's normal DNS. You lose tailnet-wide ad-blocking but keep the pretty URLs. (Chapter 8 discusses how this DNS choice interacts with VPNs when you're away from home.)

Part C: Use it and trust the cert

Step 6: Try it

From your laptop or phone, anywhere on the tailnet:

```
https://pihole.home     # Pi-hole admin (Caddy redirects to /admin)
https://abs.home        # Audiobookshelf
```

No ports, no IPs. To confirm resolution, `ping pihole.home` should answer from the Pi's `100.x.y.z`. Until you do Step 7, your browser will warn that the certificate isn't trusted (it's Caddy's private CA), that's expected, and Step 7 removes the warning for good.

Step 7: Trust Caddy's certificate (so bare names *just work*)

Without this step, typing a bare `pihole.home` is frustrating: browsers try **`https://` first**, and because they don't yet trust Caddy's private CA, they throw a "not secure" warning (or fall back to treating the name as a web search). The fix is to install **Caddy's root certificate** on each device, once. After that, the browser's automatic HTTPS upgrade lands on a *trusted* cert and bare `home.home` / `pihole.home` / `abs.home` open instantly, padlock and all.

First, export the root certificate from the Caddy container (on the Pi):

```
docker exec caddy cat /data/caddy/pki/authorities/local/root.crt \
  > ~/proxy/caddy-root-ca.crt
```

Now get that file onto each device. Since every device is already on your tailnet, the tidiest way is **Taildrop**, Tailscale's built-in file send between your own machines, no AirDrop, no email, no cable:

```
# from the Pi (or any machine that has the file), send to a peer by its
name:
tailscale file cp ~/proxy/caddy-root-ca.crt <laptop-name>:
tailscale file cp ~/proxy/caddy-root-ca.crt <iphone-name>:
```

Find the exact peer names with `tailscale status`. (If the Pi answers `Access denied: file access denied`, run `sudo tailscale set --operator=$USER` once so the Tailscale CLI works without `sudo`, then retry, or just send from a machine where it already works.)

Then install `caddy-root-ca.crt` on each device:

- **Linux laptop (system-wide):** receive the Taildrop file (it lands in your Downloads, or run `tailscale file get .`), then:

```
sudo cp caddy-root-ca.crt
/usr/local/share/ca-certificates/caddy-root-ca.crt
sudo update-ca-certificates
```

Firefox keeps its **own** trust store: Settings → *Privacy & Security* → *Certificates* → **View Certificates** → *Authorities* → **Import** the file and tick "Trust this CA to identify websites."

- **iPhone:** open the **Tailscale** app, tap the received `caddy-root-ca.crt`, and **Save to Files**. Open it from the **Files** app → iOS says "Profile Downloaded" → **Settings → General → VPN & Device Management → Install** the profile. Then (this second step is the one people miss) **Settings → General → About → Certificate Trust Settings** and toggle **on** "Enable Full Trust" for the Caddy Local Authority. (AirDrop or email work too, but Taildrop keeps it on the tailnet.)

Now reload `https://home.home`, the warning is gone, and typing the bare name works.

> **i** Why a private CA instead of a real certificate
>
> A real, publicly-trusted certificate requires a domain **you own** and a public CA (Let's Encrypt) that can verify it, which can't be done for a private `.home` name. Caddy's internal CA sidesteps that entirely: it mints certificates locally and you tell *your* devices to trust *that* CA. The trade-off is the one-time install above. (If you ever buy a real domain, point a subdomain at the Pi and drop `tls internal`, Caddy will fetch a real certificate automatically, and you can remove the root cert from your devices.)

The pattern you'll reuse

Every service from here on gets a pretty URL the same way, three small steps:

1. Put its container on the `homelab` network.
2. Add a block to `~/proxy/Caddyfile`: `name.home { tls internal\n reverse_proxy container:PORT }`.
3. Add a `name.home → 100.x.y.z` record in Pi-hole, then `docker compose restart caddy` (in `~/proxy`).

Because you've already trusted Caddy's CA, the new name gets a trusted cert automatically, no per-service certificate work. Chapter 5 does exactly this for the dashboard, giving it `https://home.home`.

Caveats

- **One-time CA trust per device.** The padlock only appears on devices where you've installed Caddy's root cert (Step 7). A brand-new device shows the warning until you do. This is the cost of a private TLD; it's a one-time step.
- **Exit nodes break `.home`.** A Mullvad exit node routes DNS through Mullvad and bypasses Pi-hole, so `*.home` won't resolve while one is active. This tension is the subject of Chapter 8; switch the exit node off (or use your home Pi as the exit node) to use the names.

Recap

- **Shared `homelab` network** so Caddy can reach services by name.
- **Pi-hole** moved its web UI to `8081` and now listens on all interfaces (LAN + tailnet).
- **Caddy** is the single front door on ports 80/443, serving HTTPS with its own internal CA and routing `pihole.home` and `abs.home` to the right containers.
- **Pi-hole DNS records + the Tailscale Override** make those names resolve on every device, everywhere.
- **Trusting Caddy's root CA** on each device makes bare names open over HTTPS with no warning and no scheme.

You now have clean, portable URLs, and a repeatable recipe for every service to come. Next, Chapter 5 adds a dashboard that ties them all together behind `https://home.home`.